APPSHEET APP ANALYSIS REPORT

เผ่าปัญญา ทรานสปอร์ต

รายงานวิเคราะห์โครงสร้างแอปและแผนปรับปรุงฉบับสมบูรณ์

ผลการวิเคราะห์เชิงลึกจากเอกสาร Application Documentation จำนวน 1,946 หน้า ครอบคลุมตาราง คอลัมน์ สูตร หน้าจอ ปุ่มสั่งงาน ระบบกรองสิทธิ์ และระบบอัตโนมัติทั้งหมดของแอป พร้อมแผนปรับปรุง 3 ระยะ และคู่มือแก้ไขรายตารางจำนวน 90 ไฟล์ที่เขียนขั้นตอนไว้จริงทุกจุด

90	2,304	115
ตาราง	คอลัมน์	หน้าจอ (Views)

214	30
ปุ่มสั่งงาน (Actions)	Bots อัตโนมัติ

PHAOPANYA TRANSPORT · วิเคราะห์เมื่อ 25 ตุลาคม 2569 · จากข้อมูล REPO PHAOPANYA_TRANSPORT

EXECUTIVE SUMMARY

บทสรุปผู้บริหาร

แอป **เผ่าปัญญา ทรานสปอร์ต** เป็นระบบบริหารงานขนส่งที่สร้างด้วย AppSheet บน Google Sheets ใช้งานจริงโดยทีมงานในทุกขั้นตอน ตั้งแต่รับงานประจำวัน สแกนเอกสาร อนุมัติราคา คิดค่าขนส่งตามระยะทาง ไปจนถึงเบิกส่วนต่างและพิมพ์ใบวางบิลแยกตามคลังปลายทาง จากการตรวจโครงสร้างทั้ง 90 ตารางและ 2,304 คอลัมน์ ทีมวิเคราะห์ขอสรุปตรง ๆ ว่า **นี่คือแอปที่ทำได้ดีกว่าที่ผู้สร้างเคยคิดไว้เยอะมาก** — โครงหลักเข้าใจงานจริงของธุรกิจ มีระบบสิทธิ์แบบสองชั้นที่หลายแอปมืออาชีพยังไม่ได้ทำ และมีระบบอัตโนมัติ 30 ตัวที่ทำงานหนักแทนคน คะแนนเฉลี่ยของตารางข้อมูลหลักอยู่ที่ **81.2 / 100** ซึ่งเป็นฐานที่ดีมากสำหรับการต่อยอด

จุดที่ทำให้คะแนนยังไม่เข้าใกล้เต็มมีสามเรื่องใหญ่ ๆ หนึ่ง มีตารางเก็บ**ราคาและข้อมูลพนักงานถึง 9 ตารางที่ยังไม่มี Security Filter** ทำให้ผู้ใช้ทุกคนโหลดและเห็นข้อมูลของกันและกันได้ทั้งหมด สอง มี**สูตรฝังตัวเลขราคาเอาไว้ข้างใน** เมื่อราคาเปลี่ยนต้องไล่แก้สูตรทีละจุด เสี่ยงคิดเงินผิด และสาม มีการใช้สูตร SELECT() ถึง 293 ครั้ง ที่ทำงานหนักขึ้นเรื่อย ๆ เมื่อข้อมูลโต ทำให้แอปจะ**เริ่มช้าในอีกไม่กี่เดือนข้างหน้า**ถ้าไม่เก็บกวาด ทั้งสามเรื่องแก้ได้ด้วยแบบแผนที่แอปมีอยู่แล้ว ไม่ต้องสร้างอะไรใหม่จากศูนย์

90	2,304	192	81.2
ตาราง (หลัก 29 · ระบบ Bot 60)	คอลัมน์ (Virtual 171)	คอลัมน์ที่มีสูตร	คะแนนเฉลี่ยตารางหลัก

520	30
จุดปรับปรุงที่ตรวจพบ	Bots ที่ทำงานอัตโนมัติ

💪 จุดแข็งที่ตรวจพบ (ควรคงไว้)

- **ระบบสิทธิ์สองชั้น** USERROLE + Security Filter คุมเพิ่ม/แก้/ลบและคุมการมองเห็นแยกกัน มีใช้อยู่ 7 ตารางแล้ว
- **วัฒนธรรมงานภาพถ่าย** — คอลัมน์รูปภาพถึง 536 จุด ใช้เป็นหลักฐานส่งของ/เอกสารคืน/ใบเสร็จน้ำมันครบทุกขั้น
- **ปุ่มลบถามยืนยันทุกปุ่ม** และ USER ทั่วไปลบข้อมูลไม่ได้เลยตาม Update mode
- **ความสัมพันธ์ Ref ถูกต้อง 100%** ทั้ง 80 จุดเชื่อม ชี้เป้าหมายถูกตารางทุกจุด
- **ระบบอัตโนมัติหนาแน่น** — พิมพ์ใบวางบิลตามคลัง คิดค่าขนส่งจาก Google Maps API และแจ้งเตือน Telegram ทำเองหมด

🔧 จุดที่ควรแก้ (เรียงตามผลกระทบ)

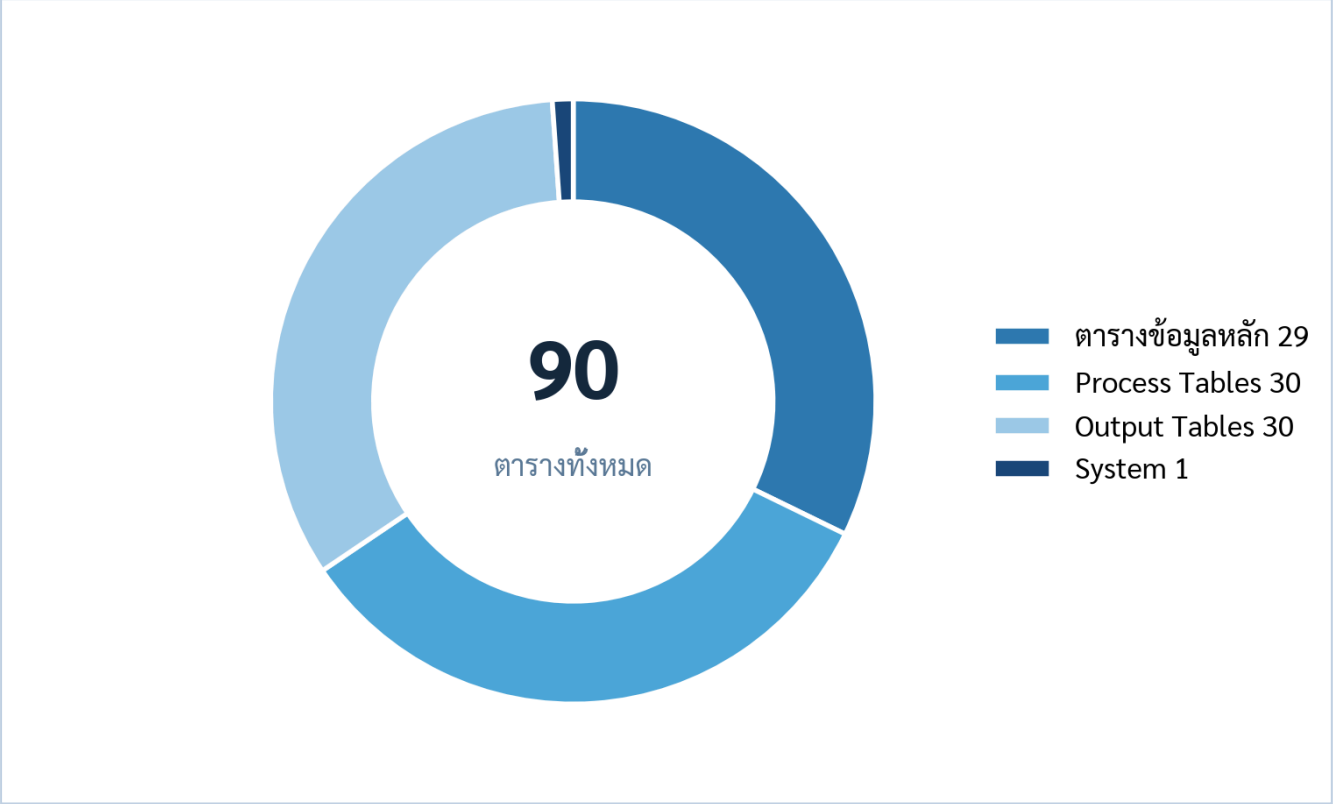
- **ความปลอดภัย:** 9 ตารางเก็บราคา/ข้อมูลพนักงาน ยังไม่มี Security Filter
- **ความเร็ว:** SELECT()/FILTER() ใน Virtual Column ของตารางใหญ่ เช่น T20 ทำเบิกส่วนต่าง (90 VC)
- **ความยืดหยุ่นราคา:** สูตรฝังราคา 50/100/300/500/1,200/2,400 ไว้ข้างใน
- **คุณภาพข้อมูล:** จำนวนเงิน 5 คอลัมน์เก็บเป็น Text, สกุลเงิน $ 2 คอลัมน์, คีย์ให้กรอกเอง 63 จุด
- **เอกสารในตัวแอป:** คอลัมน์ไม่มีคำอธิบายถึง 366 จุด — กลับมาแก้ทีหลังจะจำไม่ได้ว่าอะไรคืออะไร

ส่วนท้ายของรายงานฉบับนี้คือ **แผนปรับปรุง 3 ระยะ**ที่จัดลำดับให้แล้วว่าอะไรควรทำก่อน-หลัง และมี**คู่มือแก้ไขรายตาราง 90 ไฟล์**แนบไปกับ repo ในโฟลเดอร์ improvement/ ทุกไฟล์อธิบายปัญหา เหตุผล และขั้นตอนกดปุ่มใน AppSheet Editor เป็นภาษาไทยจบในตัว ผู้ไม่มีพื้นฐานเขียนโปรแกรมก็ทำตามได้

PART 1 · SYSTEM OVERVIEW

1. ภาพรวมโครงสร้างระบบ

โครงสร้างของแอปแบ่งเป็นสี่กลุ่มชัดเจนตามบทบาท กลุ่มแรกคือ**ตารางข้อมูลหลัก 29 ตาราง**ที่เก็บข้อมูลจริงของธุรกิจ ทั้งงานประจำวัน การวางบิล ราคา พนักงาน รถ และเอกสาร อีกสองกลุ่มคือ**Process Table และ Output Table อย่างละ 30 ตาราง** ที่ AppSheet สร้างให้อัตโนมัติเพื่อเลี้ยงระบบ Bot 30 ตัว และกลุ่มสุดท้ายคือตารางระบบ `_Per User Settings` รูปแบบนี้บอกว่าแอปไม่ใช่ "Google Sheets ที่มีหน้าจอ" แต่เป็นระบบที่มีส่วนของเครื่องจักรอัตโนมัติชัดเจน งานซ้ำ ๆ เช่น การพิมพ์ใบวางบิลแยกตามคลังปลายทาง ถูกสร้างเป็น Bot ไว้ 12 ตัว ไม่ต้องมานั่งกรองมืออีกต่อไป



ภาพที่ 1 — สัดส่วนตารางแยกตามบทบาท: ข้อมูลจริง 29 ตาราง กับตารางระบบของ Bot 60 ตาราง

ถ้ามองแผนผังความสัมพันธ์ (Ref) ทั้ง 80 จุดเชื่อม จะเห็นว่าข้อมูลไหลมารวมกันที่ตารางกลางเพียงไม่กี่ตัว **ข้อมูลการวางบิล** คือศูนย์กลางที่แท้จริง ถูกอ้างอิงจากตารางอื่นถึง 36 จุด (ส่วนใหญ่เป็นตารางระบบของ Bot ที่คำนวณจากมัน) ตามด้วย **รับงานคืนได้เงิน200** (22 จุด), **ข้อมูลพนักงาน** (21 จุด) และ **ตารางงานประจำวัน** (17 จุด) ผลพวงคือ: การแก้โครงสร้างคอลัมน์คีย์หรือเปลี่ยนชื่อของสี่ตารางนี้ กระทบกระเพื่อมกว้างกว่าที่คิด จึงควรทำด้วยความระมัดระวังและทดสอบทันที ในขณะที่ตารางรอบนอกอย่าง Announcements หรือ Acknowledge_Logs แก้ได้อิสระกว่ามาก

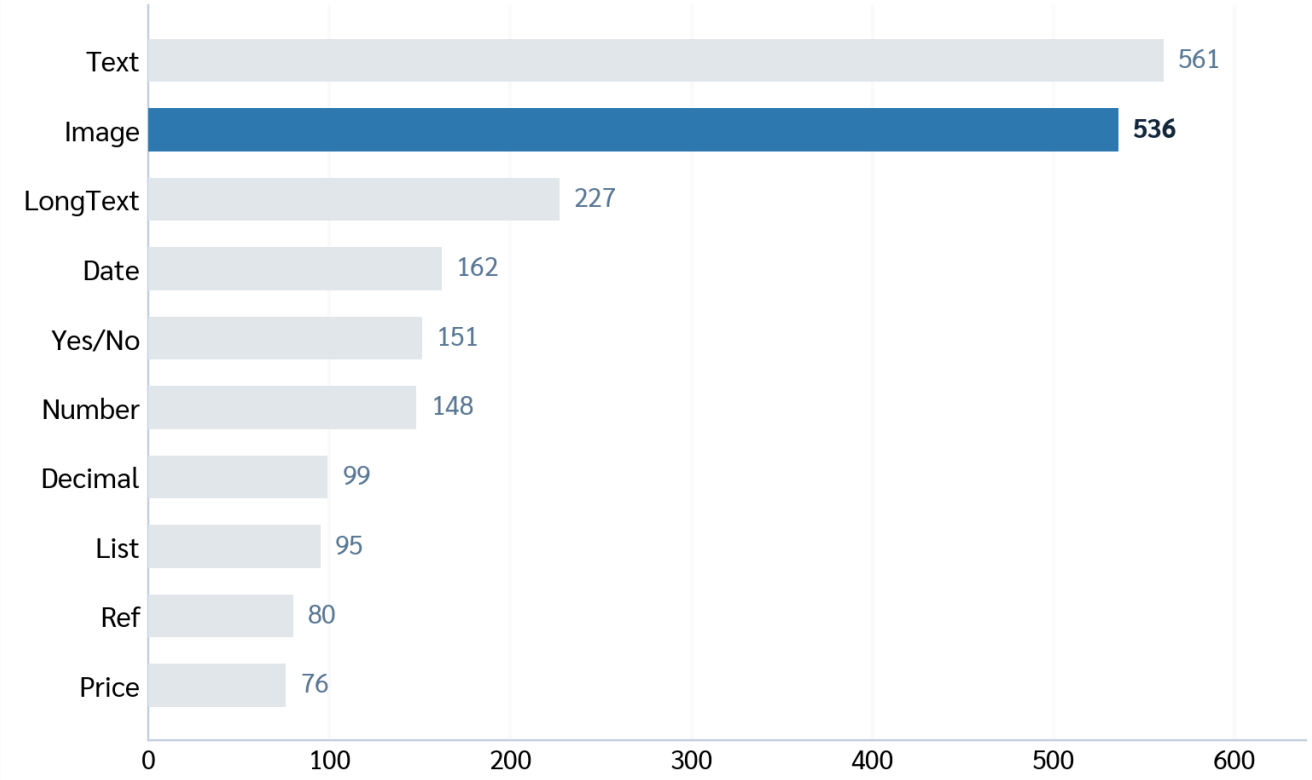
ตารางศูนย์กลาง (Hub)	ถูกอ้างอิงเข้ามา	บทบาทในระบบ
ข้อมูลการวางบิล	36	แหล่งรายการส่งของที่ Bot คำนวณราคา/สรุปยอดจาก
รับงานคืนได้เงิน200	22	ระบบงานพิเศษ 200 บาท ที่ Bot จัดเส้นทางให้อัตโนมัติ
ข้อมูลพนักงาน	21	มาสเตอร์คน เชื่อม Email เข้ากับทุกระบบสิทธิ์
ตารางงานประจำวัน	17	ใบงานรายวันของพนักงาน จุดเริ่มของเกือบทุก workflow
SCGนครหลวงJWDภูมิภาค	13	มาสเตอร์เส้นทาง/จุดส่งที่ Bot จัดเรียงและคิดระยะทาง

PART 2 · DATA MODEL

2. โครงสร้างข้อมูลและชนิดคอลัมน์

การที่แอปใช้งานได้ดีหรือช้า-พังง่ายในระยะยาว เริ่มจาก "ชนิดของคอลัมน์" เป็นเรื่องแรกเสมอ ในแอปนี้ 2,304 คอลัมน์ กลุ่มใหญ่คือข้อความ 561 คอลัมน์ และ**รูปภาพ 536 คอลัมน์** ซึ่งสะท้อนวิธีทำงานของทีมที่ใช้รูปถ่ายเป็นหลักฐานในทุกจุด — จุดนี้

ถือเป็นจุดแข็งเชิงระบบควบคุมงานจริง ๆ นอกจากนี้มีคอลัมน์ที่ตั้งค่ามาอย่างดีอีกจำนวนมาก เช่น Price 76 คอลัมน์ที่แสดงสัญลักษณ์บาทถูกต้องส่วนใหญ่ Email 29 คอลัมน์ที่ใช้เชื่อมกับระบบสิทธิ์ และ LatLong 35 คอลัมน์ที่จับพิกัดตอนกดบันทึกงาน



ภาพที่ 2 — ชนิดของคอลัมน์ 10 กลุ่มใหญ่ (รูปภาพเน้นสี = หัวใจของแอปที่ใช้พิสูจน์การทำงาน)

วิกฤต **ตารางเก็บข้อมูลอ่อนไหว 9 ตารางโดยไม่มี Security Filter**

ตารางกลุ่มนี้เก็บราคาขาย ราคาน้ำมัน ข้อมูลพนักงาน และสรุปยอดเงิน แต่ทุกผู้ใช้โหลดทุกแถวมาที่เครื่องตัวเองตอน sync — รายละเอียดและสูตรที่ใช้แก้ อยู่ในหัวข้อ 4 ความปลอดภัย

สำคัญ **จำนวนเงินบางคอลัมน์เก็บผิดชนิดข้อมูล**

พบ 5 คอลัมน์เก็บเงินเป็น Text (รวม-เรียงลำดับไม่ได้ เช่น คอลัมน์ราคาในตาราง T22 สร้างเอกสาร) และ 1 คอลัมน์เป็น Number (`ราคาตามบิล` ในข้อมูลการวางบิล — ควรเปลี่ยนเป็น Price เพื่อคุมทศนิยมและแสดง ฿) อีก 2 คอลัมน์ (`ราคารวม` , `ราคารวมคนขับ` ในสร้างเอกสาร) ตั้งสกุลเงินเป็นดอลลาร์ $ แทนที่จะเป็นบาท แก้ใน Type Details ได้ใน 2 นาที

สำคัญ **คอลัมน์ Key ให้ผู้ใช้กรอกเอง 63 จุด**

คีย์คือหมายเลขประจำแถวที่ระบบความสัมพันธ์ทั้งหมดพึ่งพา ถ้าพนักงานพิมพ์ซ้ำหรือลืมกรอก ระบบจะเตือนตอนบันทึกและงานสะดุดทันที คีย์ที่มีความหมายธุรกิจอยู่แล้ว (เช่น เลข Invoice) ให้คงไว้แต่เปิด Required ส่วนคีย์ที่เป็นแค่หมายเลขเทคนิค ให้ตั้ง Initial Value เป็น `UNIQUEID()` ระบบจะสร้างให้เอง ซ้ำไม่ได้แน่นอน — รายชื่อครบทุกจุดอยู่ในคู่มือรายตาราง

ตรวจสอบ **ตาราง "ข้อมูลงานRT" ไม่มีคอลัมน์ในเอกสาร**

ตารางนี้ถูก Ref มาถึง 4 จุดและมีสูตรใช้งาน แต่เอกสาร export มาว่างเปล่า 0 คอลัมน์ — ควรเปิด AppSheet Editor ตรวจว่าโครงสร้างจริงเป็นอย่างไร ถ้าตารางไม่มีข้อมูลจริง แสดงว่าเอกสารอาจ export ไม่ครบ ถ้าไม่ได้ใช้แล้วก็ลบให้สะอาด

3. สูตร ความสัมพันธ์ และความเร็วของแอป

แอปนี้มีสูตรหนาแน่นมาก: 192 คอลัมน์มี App Formula, 127 คอลัมน์มี Initial Value และ 288 คอลัมน์มี Valid_If คอยตรวจความถูกต้องก่อนบันทึก ฟังก์ชันที่ใช้บ่อยที่สุดคือ `SELECT()` ถึง 293 ครั้ง, `LOOKUP()` 125 ครั้ง และ `FILTER()` 18 ครั้ง ทั้งสามตัวนี้เป็น "ฟังก์ชันสแกนตาราง" คือต้องไล่อ่านแถวของตารางปลายทางทุกครั้งที่คำนวณ — ตอนข้อมูลมี 100 แถวไม่รู้สึกอะไร แต่พอเป็น 5,000 แถว ทุกหน้าจอที่เปิดจะเริ่มหนักขึ้นเรื่อย ๆ โดยเฉพาะเมื่ออยู่ใน Virtual Column ที่คำนวณซ้ำทุกครั้งที่เปิดหน้า

ฟังก์ชัน	จำนวนครั้ง	ความหมาย / ข้อสังเกต
`SELECT()`	293	ดึงค่าหลายแถวจากตาราง — หนักสุด ควรแคบเงื่อนไขหรือเปลี่ยนเป็น Ref/LOOKUP ให้มากที่สุด
`SPLIT()` + `TEXT()`	155 / 164	ใช้แยกค่าจากข้อความเยอะ ชี้ว่าข้อมูลบางช่องเก็บหลายค่าในช่องเดียว
`LOOKUP()`	125	หาค่าเดียวจากตาราง — 89 จุดเขียนให้เร็วขึ้นได้ด้วยการอ้างผ่าน Ref แบบจุด
`UNIQUEID()`	28	สร้างคีย์อัตโนมัติ — ควรขยายไปใช้อีก 35 จุดที่ยังให้ผู้ใช้กรอกคีย์เอง
`USEREMAIL()/USERROLE()`	45 / 33	ผูกข้อมูลกับผู้ใช้ — ถูกที่ถูกที่ส่วนใหญ่ มี 2 จุดที่ควรทบทวน
`TODAY()`	51	วันที่ปัจจุบัน — ระวังงานที่ทำข้ามเที่ยงคืน ควรกันช่วงด้วย TODAY()-1
`ENCODEURL()`	25	สร้างลิงก์ Google Maps/แผนที่นำทาง ใช้งานจริงกับงานขนส่งได้ดีมาก

กรณีศึกษา 1 — ตารางทำเบิกส่วนต่างSCgวังน้อย (T20): เครื่องคำนวณที่หนักที่สุดของแอป

ตารางนี้มี 136 คอลัมน์ ในนั้นเป็น Virtual Column ถึง 90 คอลัมน์ และมี LOOKUP ถูกเรียกใน 81 คอลัมน์ แปลเป็นภาษาคนคือทุกครั้งที่เปิดแถวเดียวของตารางนี้ แอปต้องไล่อ่านตารางอื่นหลายสิบรอบ เช่น คอลัมน์ "เรท น้ำมันดีเซล" เดียว ซ้อน SELECT ใน SELECT ถึงสองชั้นเพื่อหาราคาน้ำมันตามวันที่และชื่องาน (ดูโครงสร้างจริงข้างล่าง) แนวทางแก้ไม่ใช่การลบความฉลาดออก แต่คือ**ทำให้ผลลัพธ์ถูกเก็บเป็นค่าจริง** โดยให้ Bot คำนวณตอนบันทึกงานแล้วเขียนลง Google Sheets จากนั้นหน้าจออ่านค่าสำเร็จรูป แอปจะเร็วขึ้นมากทันที และถ้าอยากตรวจสอบย้อนหลังยังมีตัวเลขจริงเก็บอยู่เป็นหลักฐาน

```
IF( ISBLANK([_THISROW].[วันที่ส่งสินค้า]), "",
  IF( ISNOTBLANK( ANY( SELECT( PriceOverride[UsePriceDate],
        AND( [StartDate] <= [_THISROW].[วันที่ส่งสินค้า],
             [EndDate]   >= [_THISROW].[วันที่ส่งสินค้า],
             [JobName]   =  [_THISROW].[ชื่องาน] ) ) ) ),
    ANY( SELECT( ราคาน้ำมันดีเซล[ราคา/หน่วย], ... ) ),
    ... ))   ← สูตรจริงจากคอลัมน์ "เรท น้ำมันดีเซล" (ตัดตอน)
```

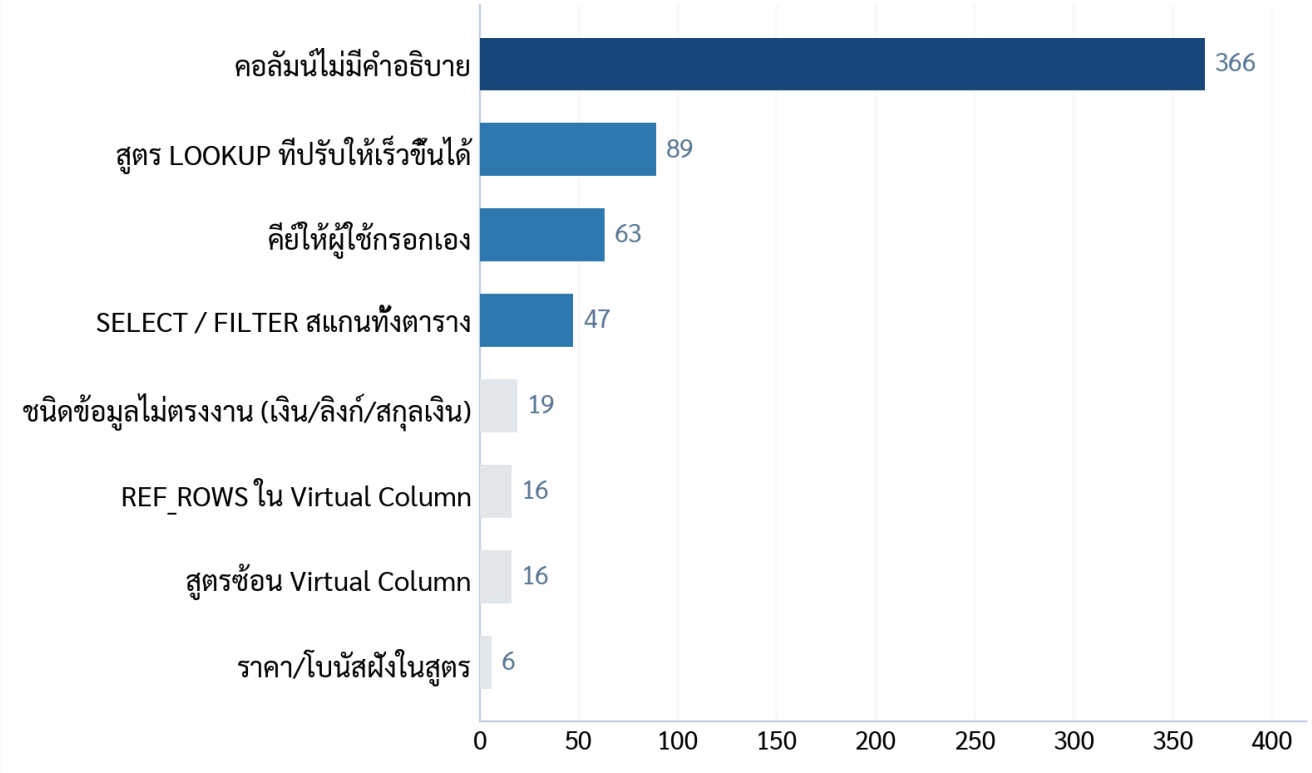
กรณีศึกษา 2 — ราคาฝังอยู่ในสูตร: แก้ราคาทีหลังยากและเสี่ยงคิดผิด

คอลัมน์ "ราคาจุดเพิ่ม" และ "ราคาเพิ่ม" ในข้อมูลการวางบิล ฝังราคาของแต่ละงานไว้ในสูตรโดยตรง พอมีการปรับราคาหรือเพิ่มงานใหม่ ต้องมาแก้สูตรนี้ให้ถูก และไล่หาว่าสูตรอื่นที่คัดลอกไปใช้อีก (เช่นใน T20) ยังฝังราคาชุดเก่าอยู่หรือเปล่า ทั้งที่แอปมีตาราง **Price_Table** และ **PriceOverride** ไว้สำหรับงานนี้อยู่แล้ว วิธีแก้คือย้ายราคาทุกตัวเข้าตารางราคาให้ครบทุกเงื่อนไข แล้วให้สูตรไป LOOKUP จากตารางแทนเลขที่ฝังไว้ จุดนี้เป็นการลงทุนเวลาครั้งเดียวที่ตอบแทนทุกครั้งที่ราคาเปลี่ยนในอนาคต

```
IFS( AND([ชื่องาน]="เบทาโกร+เหมาตู้", [จำนวนจุดรับ / จุดส่ง] >= 11), 300,
  ... )   ← ราคา 50 / 100 / 250 / 300 ฝังในสูตร "ราคาจุดเพิ่ม" (ตัดตอน)
```

สำคัญ **สูตรซ้อน Virtual Column 16 จุด และสูตรยาวเกิน 7 จุด**

Virtual Column ที่อ้าง Virtual Column อื่นต่อเป็นทอด ทำให้แอปคำนวณซ้ำหลายชั้นโดยไม่จำเป็น แนะนำคัดลอกสูตรต้นทางมาใส่ตรง ๆ หรือรวมเป็นคอลัมน์เดียว — รายการคอลัมน์ครบทุกจุดอยู่ในคู่มือรายตาราง (สัญลักษณ์ 🟡)



ภาพที่ 3 — จุดปรับปรุงทั้ง 520 จุด จัดกลุ่มตามประเภท (สีเข้ม = กลุ่มที่พบมากที่สุด)

PART 4 · SECURITY & PRIVACY

4. ความปลอดภัยและสิทธิ์การเข้าถึง

โครงสร้างสิทธิ์ของแอปนี้ออกแบบมาดีเป็นสองชั้น ชั้นแรกคือ **Update mode ต่อตาราง** ที่ใช้สูตร `SWITCH(USERROLE(), "ADMIN", "ALL_CHANGES", "USER", "ADDS_AND_UPDATES", "READ_ONLY")` แปลว่า ADMIN ทำได้ทุกอย่าง USER เพิ่มและแก้ได้แต่ลบไม่ได้ และบทบาทอื่นอ่านอย่างเดียว ชั้นที่สองคือ **Security Filter** ที่กรองแถวตั้งแต่ระดับแหล่งข้อมูล เช่น `=OR(USERROLE() = "ADMIN", [Email พนักงาน] = USEREMAIL())` คือเห็นเฉพาะงานของตัวเอง ยกเว้น ADMIN เห็นทุกแถว ปัจจุบันมีใช้อยู่ 7 ตาราง และมี View 5 จุดที่ซ่อนหน้าจอสำหรับ ADMIN เท่านั้นด้วย Show_If — นี่คือแบบแผนสำเร็จที่แค่ "ก๊อปต่อ" ก็ใช้ได้

วิกฤต **9 ตารางเก็บราคา/ข้อมูลพนักงาน แต่ไม่มี Security Filter**

ผู้ใช้ทุกคนโหลดทุกแถวมาที่เครื่องตัวเองตอน sync แปลว่าเปิดดูยอด ราคา และข้อมูลของเพื่อนร่วมทีมได้ทั้งหมด — ทั้งเรื่องความเป็นส่วนตัวและความอึดอัดในทีม วิธีแก้คือคัดลอกสูตร filter มาตรฐานของแอปไปใส่ 9 ตารางนี้ (ทดสอบด้วยบัญชี USER จริง 1 บัญชีหลังใส่ทุกครั้ง)

ข้อมูลพนักงาน | PDFพนักงาน | ราคาน้ำมันดีเซล | ราคาในระบบก่อนส่งสินค้า | Price_Table | PriceOverride | สรุปเบิกส่วนต่าง | สรุป_เจ้าของสินค้า | สรุป_Shipment

หมายเหตุ: ตารางอีก 13 ตารางที่ไม่มี filter เป็นตารางที่ทุกคนในทีมควรเห็นเหมือนกัน (เช่น ประกาศ หน้าหลัก ตารางราคาที่ใช้ร่วม) ถือว่าเหมาะสม แต่ควรจดไว้ว่า "ห้ามเพิ่มข้อมูลส่วนบุคคลในตารางเหล่านี้ภายหลัง"

สำคัญ **ข้อมูลส่วนบุคคลในเอกสารและ repo**

เอกสารที่ใช้วิเคราะห์เคยมีอีเมลพนักงานจริง 14 รายการและเบอร์โทร 2 หมายเลข ซึ่งถูกปกปิดเป็น placeholder ทั้งหมดก่อนขึ้น GitHub แล้ว (ตรวจซ้ำแล้วเหลือ 0) ส่วน Google Sheets จริงที่เก็บข้อมูลลูกค้า ยังควรเปิดการแชร์แบบเลือกคนเท่านั้น และเนื่องจาก GitHub token ถูกแชร์ในแชท แนะนำให้ **เปลี่ยน (revoke) token หลังงานชุดนี้เสร็จ** เป็นนิสัยที่ดีต่อความปลอดภัยบัญชี

ทำไว้ดีแล้ว **ปุ่มลบและสิทธิ์การลบ**

ตรวจปุ่มลบทั้ง 29 ปุ่ม (1 ปุ่มต่อตารางหลัก) พบว่าเปิด "Needs Confirmation" ครบทุกปุ่ม และ USER ทั่วไปไม่มีสิทธิ์ลบข้อมูลเลยตาม Update mode — โครงสร้างกันกดพลาดระดับนี้เป็นมาตรฐานที่ดี ควรคงไว้ตลอดเมื่อสร้างตารางใหม่

PART 5 · VIEWS & ACTIONS

5. หน้าจอ ปุ่มกด และประสบการณ์ใช้งาน

หน้าจอทั้ง 115 หน้า กระจายเป็นหน้ารายละเอียด 38, ฟอร์มกรอก 36, ตาราง 32, การ์ด (Deck) 8 และแกลเลอรีรูป 1 โครงสร้างพื้นฐานครบ แต่มีสัดส่วนที่น่าสังเกต: **Table view ยังเป็นกลุ่มใหญ่ที่สุดของหน้า "รายการ"** ขณะที่ Deck ซึ่งอ่านงานได้เร็วกว่ามากบนมือถือ (การ์ดใหญ่ มีรูป เรียงตามกลุ่ม) ยังมีแค่ 8 หน้า การเปลี่ยนหน้ารายการหลักของพนักงาน เช่น "ตารางงานประจำวัน" จาก Table เป็น Deck ที่จัดกลุ่มตามวันที่/สถานะ คือการปรับปรุงที่แทบไม่เสียค่าใช้จ่ายแต่พนักงานรู้สึกได้ทันทีทุกวัน

ด้านปุ่มสั่งงานทั้ง 214 ปุ่ม จัดกลุ่มได้เป็น: ตั้งค่าคอลัมน์ 49 ปุ่ม, เปิดหน้าจอ 29, เพิ่มแถว 29, แก้แถว 29, ลบแถว 29, ส่งออก/พิมพ์ 21, ส่งอีเมล 12 และอื่น ๆ — แสดงว่าระบบนี้ "คลิกน้อยแต่ทำงานเยอะ" ด้วยการออกแบบปุ่มช่วยกรอก จุดที่ควรเก็บรายละเอียดต่อคือ ปุ่มตั้งค่าคอลัมน์ทั้ง 49 ปุ่ม ควรตรวจว่าไม่มีปุ่มไหนไปตั้งค่าคอลัมน์ที่มี App Formula (ค่าจะถูกสูตรทับ) ซึ่งตรวจแล้วปัจจุบันยังไม่พบปัญหานี้ จึงเหลือแค่กฎไว้กันตอนแก้แอปภายหลัง

ส่วน **Format Rules มีเพียง 11 กติกา** (อยู่กับตารางข้อมูลการวางบิลเป็นหลัก) และ **Slices มีเพียง 8 ตัว** — นี่คือของสองอย่างที่ "เติมแล้วแอปดูเป็นมืออาชีพขึ้นเร็วที่สุด" เช่น กติกาสีแถวสำหรับงานที่ค้างเกิน 3 วัน หรือ Slice "งานของฉันวันนี้" ที่กรองด้วย USEREMAIL() เพื่อให้พนักงานเปิดเจองานตัวเองทันทีโดยไม่ต้องเลื่อนหา คู่มือรายตารางมีคำแนะนำตรงจุดนี้แนบไว้ท้ายแต่ละไฟล์แล้ว

PART 6 · AUTOMATION

6. ระบบอัตโนมัติ (Bots) ทั้ง 30 ตัว

จากร่องรอยตาราง Process/Output ทั้ง 60 ตาราง สรุปได้ว่าแอปมี Bot ทำงานอยู่ราว 30 ชุด จัดกลุ่มตามหน้าที่ได้เป็น **พิมพ์ใบวางบิลแยกตามคลัง/ลูกค้า 12 ตัว** (โอนคลังมโนยนต์ บางบัวทอง บุญถาวร เอเธน ไอโรบอท ไทยเปเปอร์ เบทาโกรเหมาตู้ ฯลฯ), **เบิกค่าใช้จ่าย/ส่วนต่าง 5 ตัว** (เบิกส่วนต่าง SCG วังน้อย ค่ายกขึ้นชั้น โบนัส 25% ของค่าเที่ยว ส่วนเพิ่ม 200 บาท), **จัดเส้นทาง/คิดระยะทาง 2 ตัว** ที่เรียก Google Maps Routes API, **รับสินค้าคืน/เอกสาร 3 ตัว**, **แจ้งเตือน/สื่อสาร 3 ตัว** (Telegram, KM) และโอนคลัง/สรุปงานอีก 5 ตัว นี่คือระบบที่ "ตั้งไว้ครั้งเดียว ทำงานทุกวัน" ของจริง

การดูแลรักษา **สิ่งที่ควรทำกับตารางระบบของ Bot**

ตาราง Process/Output 60 ตารางจะสะสมแถวสถานะจากการรัน Bot ทุกครั้ง ควรกำหนดรอบทำความสะอาด เช่น ทุกเดือนลบแถวเก่าเกิน 60 วัน ห้ามลบหรือเปลี่ยนชื่อคอลัมน์ระบบ (`_ComputedKey_ID_*`) เด็ดขาด เพราะจะทำให้ Bot พังทันทีโดยไม่เตือน และถ้า Bot ตัวใดเลิกใช้แล้ว ให้ลบที่ Automation ให้สะอาดเพื่อไม่ให้ตารางระบบงอกต่อ

7. แผนปรับปรุง 3 ระยะ (ลำดับที่ควรทำจริง)

แผนนี้จัดเรียงด้วยหลัก "ปิดรูรั่วก่อน แล้วค่อยเร่งเครื่อง" — ระยะแรกเน้นเรื่องที่เสี่ยงข้อมูลหรือแก้เสร็จในไม่กี่นาที ระยะสองคือความเรียบร้อยของข้อมูลที่ทำทีละตาราง และระยะสามคือเรื่องความเร็วและประสบการณ์ใช้งานที่ต้องทดสอบกับผู้ใช้จริง ทุกข้อในแผนมีขั้นตอนละเอียดพร้อมสูตรอยู่ในคู่มือรายตาราง (โฟลเดอร์ improvement/ ใน repo) แล้ว แผนนี้ทำหน้าที่เป็นเส้นทางว่าวันไหนทำอะไร

ระยะที่ 1 — ปิดความเสี่ยงด่วน · สัปดาห์ที่ 1–2 · ประมาณ 2–3 ชั่วโมง

1. เพิ่ม Security Filter ให้ 9 ตารางอ่อนไหว (คัดลอกสูตรมาตรฐานของแอป) แล้วทดสอบด้วยบัญชี USER 1 บัญชี
2. เปลี่ยนสกุลเงิน $ เป็น ฿ ในคอลัมน์ ราคารวม / ราคารวมคนขับ (T22 สร้างเอกสาร)
3. เปลี่ยน `ราคาตามบิล` (T04) จาก Number เป็น Price และ 5 คอลัมน์เงินที่เป็น Text ให้เป็น Price/Decimal
4. เปิด AppSheet Editor ตรวจตาราง ข้อมูลงานRT ที่เอกสารแสดงว่างเปล่า — เติมโครงสร้างหรือลบทิ้ง
5. ตั้ง Initial Value `UNIQUEID()` ให้คีย์เทคนิคที่ยังให้กรอกเอง (เริ่มจากตารางที่พนักงานใช้เพิ่มแถวบ่อยสุด)
6. เปลี่ยน (revoke) GitHub token เก่า และหมุนเวียน token ใหม่หลังใช้เสร็จทุกครั้ง

ระยะที่ 2 — เก็บข้อมูลและราคาให้เป็นระบบ · สัปดาห์ที่ 3–6 · ทีละตารางตามคู่มือ

1. ย้ายราคาที่ฝังในสูตร (ราคาจุดเพิ่ม, ราคาเพิ่ม, โบนัส 25%, ค่าป่ะหน้า ฯลฯ) เข้า Price_Table แล้วให้สูตร LOOKUP แทน — จุดเปลี่ยนที่ใหญ่ที่สุดของระยะนี้
2. แปลง Enum ที่มีตัวเลือกมากกว่า 12 ค่า (4 จุด) เป็น Ref ชี้ตาราง lookup เพื่อเพิ่มค่าได้โดยไม่แก้สูตร
3. เติม Description ให้คอลัมน์ 366 จุด — ทำวันละ 1–2 ตารางตามดัชนีในคู่มือ (เริ่มจากตารางที่ใช้บ่อยที่สุด: ข้อมูลการวางบิล → ตารางงานประจำวัน → ทำเบิกส่วนต่าง)
4. ทบทวนสูตร LOOKUP 89 จุด — จุดที่มี Ref อยู่แล้ว เปลี่ยนเป็นการอ้างแบบจุด `[Ref].[ฟิลด์]` ให้สั้นและเร็วขึ้น
5. ตรวจคอลัมน์อีเมล/ลิงก์ที่เก็บเป็น Text (11 จุด) เปลี่ยนเป็น Email/Url ตามงานจริง

ระยะที่ 3 — เร่งความเร็วและยกระดับประสบการณ์ · เดือนที่ 2–3 · ทดสอบกับผู้ใช้จริงทุกขั้น

1. ลดภาระ Virtual Column ในตารางหนัก: T20 ทำเบิกส่วนต่าง (90 VC), T22 สร้างเอกสาร, สรุป_Shipment — เปลี่ยนค่าที่ไม่ต้องเรียลไทม์เป็นคอลัมน์จริงที่ Bot เขียนให้
2. ทำความสะอาดตาราง Process/Output ของ Bot เก่ากว่า 60 วัน และกำหนดรอบทำความสะอาดรายเดือน
3. ยกระดับหน้ารายการหลักเป็น Deck + GroupBy (เริ่มที่ ตารางงานประจำวัน และ ข้อมูลการวางบิล) พร้อม Slice "งานของฉันวันนี้"
4. เพิ่ม Format Rules สีเตือนตามสถานะ เช่น งานค้างเกิน 3 วัน = แถวแดง, รออนุมัติ = เหลือง
5. สร้างหน้าสรุปผู้บริหาร (View ประเภท Chart) แสดงยอดส่งต่อวัน/คนขับ จากข้อมูลที่มีอยู่แล้ว
6. เปิด Deployment Check ของ AppSheet และสำรอง Google Sheets ด้วยการคัดลอกประจำเดือน

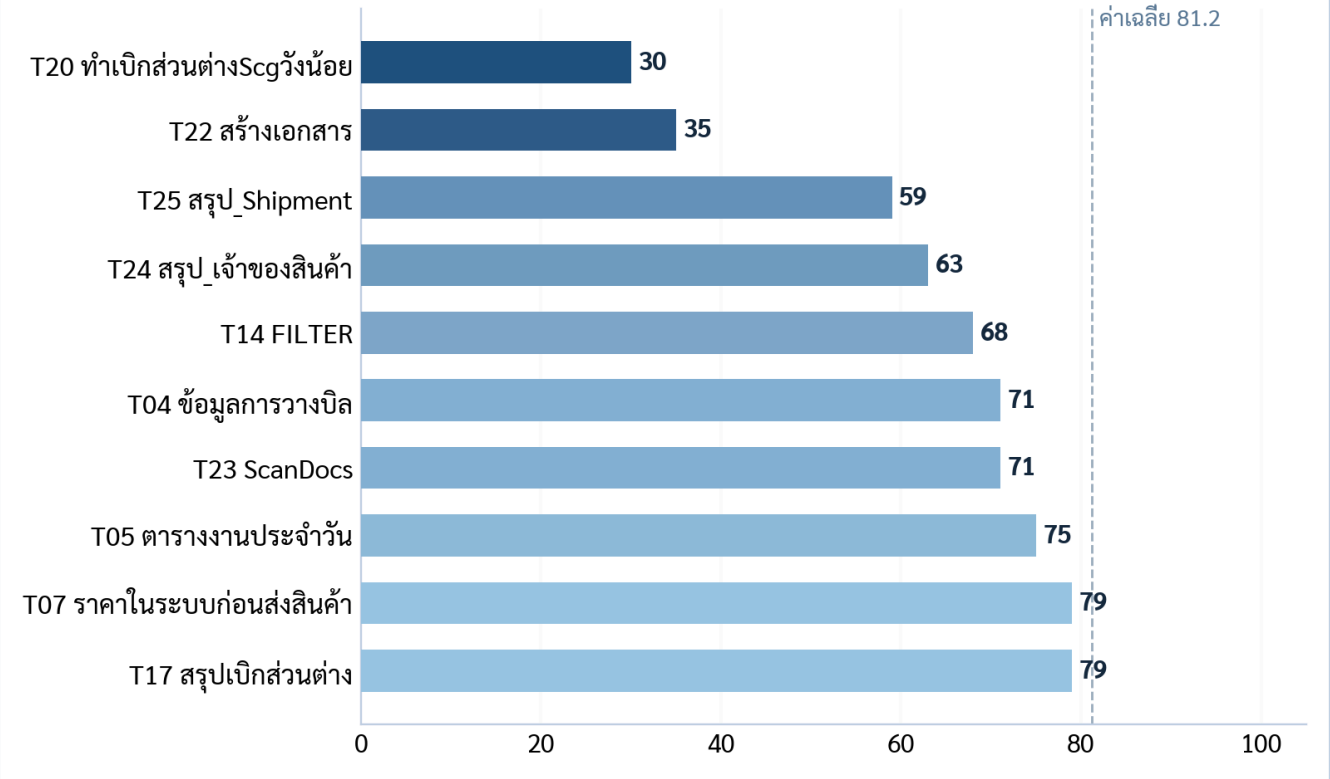
กติกาการทำงานที่แนะนำ (สำคัญพอ ๆ กับรายการข้างบน)

1. แก้ทีละ 1–2 ตารางต่อวัน อย่าแก้หลายตารางพร้อมกันแล้วทดสอบทีเดียว
2. หลังแก้โครงสร้างคอลัมน์/สูตร ทดสอบทั้งบัญชี ADMIN และ USER จริงทุกครั้ง
3. ห้ามแก้ข้อมูลใน Google Sheets ระหว่างมีคนใช้แอปอยู่ (ทำนอกเวลาทำงาน)
4. ติ๊กเช็กลิสต์ท้ายไฟล์คู่มือใน GitHub ทุกครั้งที่แก้เสร็จ เพื่อจดว่าทำถึงไหน

8. คู่มือแก้ไขรายตาราง 90 ไฟล์ — ใช้ยังไง

ทั้ง 520 จุดปรับปรุงในรายงานนี้ ถูกแตกเป็น**คู่มือตารางต่อตาราง 90 ไฟล์**ในโฟลเดอร์ `improvement/` ของ repo (ตั้งชื่อไฟล์ตรงกับโฟลเดอร์ตารางเดิม เช่น `T20_ทำเบิกส่วนต่างScgวังน้อย.md`) แต่ละไฟล์ประกอบด้วย: ข้อมูลพื้นฐานของตาราง, จุดที่ทำไว้ดีอยู่แล้ว, สรุปสิ่งที่ควรแก้เรียงตามความสำคัญ, รายละเอียดทุกปัญหาพร้อมสูตรจริงและขั้นตอนกดปุ่มใน AppSheet Editor, สถานะคอลัมน์**ครบทุกคอลัมน์**, Views/Actions/Slices/Format Rules ของตารางนั้น และเช็กลิสต์ที่ติ๊กได้เมื่อทำเสร็จ

สัญลักษณ์ที่ใช้: 🔴 **วิกฤต** = ควรแก้โดยเร็วที่สุด (เสี่ยงข้อมูลหาย/รั่วไหล), 🟠 **สำคัญ** = แก้เมื่อเริ่มปรับปรุง, 🟡 **แนะนำ** = ทำภายหลังได้ไม่เร่งด่วน และ ✅ = ผ่านการตรวจทุกข้อแล้ว — เปิดดูไฟล์บน GitHub ได้เลย และแก้ไฟล์ (ติ๊กเช็กลิสต์) ผ่านปุ่ม Edit บนหน้าเว็บได้โดยตรง ไม่ต้องมีโปรแกรมช่วย



ภาพที่ 4 — 10 ตารางข้อมูลหลักที่คะแนนต่ำสุด (เส้นประ = ค่าเฉลี่ย 81.2) เริ่มทำจากด้านบนของกราฟนี้

คะแนนต่ำสุดไม่ได้แปลว่าตารางนั้น "แย่" — ตาราง T20 ได้ 30 เพราะเป็นตารางที่ฉลาดและหนักที่สุด (200 จุดที่ควรเก็บรายละเอียด) ขณะที่ตารางที่ได้ 96–97 อย่าง ข้อมูลการเข้าบำรุงรักษา เป็นตัวอย่างโครงสร้างที่สะอาดจริง ๆ ใช้คะแนนนี้เป็น "ลำดับความจำเป็น" มากกว่า "การให้เกรด" และควรทำตามแผน 3 ระยะในหัวข้อ 7 ซึ่งจัดลำดับโดยคำนึงถึงผลกระทบก่อนเป็นหลัก

ลำดับ	ตาราง	คะแนน	สิ่งที่ไฟล์คู่มือชี้เป้าไว้
1	T20 ทำเบิกส่วนต่างScgวังน้อย	30	90 VC หนัก · LOOKUP 81 จุด · SELECT ซ้อน
2	T22 สร้างเอกสาร	35	เงินเป็น Text · สกุลเงิน $ · ราคาฝังสูตร
3	T25 สรุป_Shipment	59	ไม่มี Security Filter · สูตรซ้อน VC
4	T24 สรุป_เจ้าของสินค้า	63	ไม่มี Security Filter · คีย์กรอกเอง
5	T14 FILTER	68	สูตรซ้อน · คีย์กรอกเอง
6	T04 ข้อมูลการวางบิล	71	ราคาฝังสูตร · ราคาตามบิลเป็น Number
7	T23 ScanDocs	71	คีย์กรอกเอง · คอลัมน์ไม่มีคำอธิบาย
8	T05 ตารางงานประจำวัน	75	เดือนคำนวณแบบฝัง 543 · VC ซ้อน
9	T07 ราคาในระบบก่อนส่งสินค้า	79	ไม่มี Security Filter · ID_ราคา
10	T17 สรุปเบิกส่วนต่าง	79	ไม่มี Security Filter · ข้อมูลสรุปเงิน

ถ้าเริ่มพรุ่งนี้เช้า — ทำ 5 อย่างนี้ก่อน (ใช้เวลารวมไม่ถึง 2 ชั่วโมง)

แผนสัปดาห์แรกฉบับพกพา

1. **5 นาที** — เปลี่ยนสกุลเงิน $ เป็น ฿ ในคอลัมน์ ราคารวม และ ราคารวมคนขับ (T22 สร้างเอกสาร > Type Details > Currency Symbol)
2. **5 นาที** — เปลี่ยนชนิดคอลัมน์ ราคาตามบิล จาก Number เป็น Price (T04 ข้อมูลการวางบิล)
3. **15 นาที** — เพิ่ม Security Filter ให้ตาราง ข้อมูลพนักงาน เป็นตารางแรก แล้วทดสอบด้วยบัญชี USER จริง 1 บัญชี
4. **10 นาที** — ตั้ง Initial Value เป็น `UNIQUEID()` ให้คอลัมน์คีย์ที่ยังให้พนักงานกรอกเอง (เริ่มที่ T04 และ T05)
5. **30 นาที** — เปิดคู่มือ `T20_ทำเบิกส่วนต่างScgวังน้อย.md` อ่านจบ แล้ววางแผนลด Virtual Column 90 ตัวเป็นชุด ๆ ตามระยะที่ 3

เมื่อทำครบ 5 ข้อนี้ แอปจะปลอดภัยขึ้นทันทีในส่วนที่เสี่ยงที่สุด และตัวเลขเงินแสดงผลถูกต้องครบทุกหน้าจอ จากนั้นค่อยไล่ทำตามแผน 3 ระยะในหัวข้อ 7 โดยเปิดคู่มือรายตารางคู่ไว้เสมอ — ทุกไฟล์เขียนไว้เพื่อให้เปิดแล้วทำตามได้เลย โดยไม่ต้องเดาว่าต้องกดปุ่มไหน ทำวันละนิด สม่ำเสมอ กว่าจะครบทั้ง 90 ตาราง แอปจะอยู่ในระดับที่พร้อมขยายทีมงานได้เลย

คำตอบของคำถามที่ว่า "มันจะดีกว่านี้ได้อีกไหม" คือ — ได้ และใกล้ตัวกว่าที่คิด

แอปนี้ไม่ได้ดี "เพราะโชค" แต่ดีเพราะโครงหลักเข้าใจงานจริง ระบบสิทธิ์ทำมามาตรฐาน และระบบอัตโนมัติ 30 ตัวที่คนสร้างตั้งใจวางไว้เอง สิ่งที่เหลือคือการเก็บรายละเอียดตามแผน 3 ระยะ ซึ่งแตกเป็นขั้นตอนภาษาไทยพร้อมทำตามแล้วในคู่มือ 90 ไฟล์ — ทำวันละหน่อย ภายใน 2–3 เดือนแอปจะเปลี่ยนระดับไปเลย

`github.com/tongtanaboon-outlook/Phaopanya_Transport → improvement/`